DSA · NOTES

Binary Search

2026-10-22

Finding an item in a sorted array by halving the search space: the idea, the code, why it is $O(\log n)$, and the classic bugs.

Contents

1 The idea 2

2 The algorithm 2

2.1 Iterative version 2

3 Why it is $O(\log n)$ 3

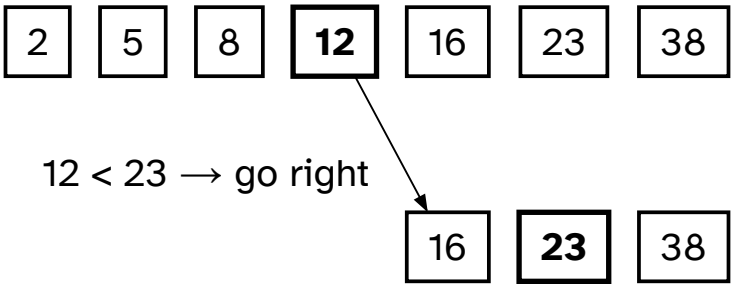
CHAPTER 1

The idea

Binary search finds a target in a **sorted** array by repeatedly halving the part of the array where the target could still be.

DEF **Search space** — The range of indices `[lo, hi]` that might still contain the answer. Every step must shrink it, and must never throw the answer away.

KEY Sorted order lets one comparison with the middle element discard **half** of the remaining elements.



Searching for 23 in a sorted array

CHAPTER 2

The algorithm

2.1 Iterative version

```
def binary_search(a, target):
    lo, hi = 0, len(a) - 1
    while lo <= hi:         # space non-empty
        mid = lo + (hi - lo) // 2
        if a[mid] == target:
            return mid
        elif a[mid] < target:
            lo = mid + 1    # drop left half
        else:
            hi = mid - 1    # drop right half
    return -1
```

⚠ CAREFUL `mid = (lo + hi) // 2` can overflow in C/Java when `lo + hi` exceeds the integer limit. Use `lo + (hi - lo) // 2`.

⚠ CAREFUL `while lo < hi` with `hi = mid - 1` skips the last element. Match the loop condition to how you shrink the range.

CHAPTER 3

Why it is $O(\log n)$

After k steps the search space has $\frac{n}{2^k}$ elements. It stops when only one is left:

STEPS

$$\frac{n}{2^k} = 1 \quad \Longrightarrow \quad k = \log_2 n$$

Case	Time	Space
Iterative	$O(\log n)$	$O(1)$
Recursive	$O(\log n)$	$O(\log n)$ stack

Example: For $n = 10^6$ you need at most $\lceil \log_2 10^6 \rceil = 20$ comparisons. A linear scan could need $1,000,000$.

MY NOTE *Binary search works on any **monotonic** yes/no condition, not only on arrays.*

Q What must be true of the input?
A It must be sorted, or more generally the condition must be monotonic: all "no" answers come before all "yes" answers.

AT A GLANCE

- Sorted input → compare with the middle → discard half.
- Loop while `lo <= hi`; move with `mid ± 1`.
- $O(\log n)$ time, $O(1)$ space when iterative.